



CentraleSupélec



Markov Decision Processes

Yiping Fang, Professor

Chair on Risk and Resilience of Complex Systems,

Laboratoire Génie Industriel, CentraleSupélec, Université Paris Saclay

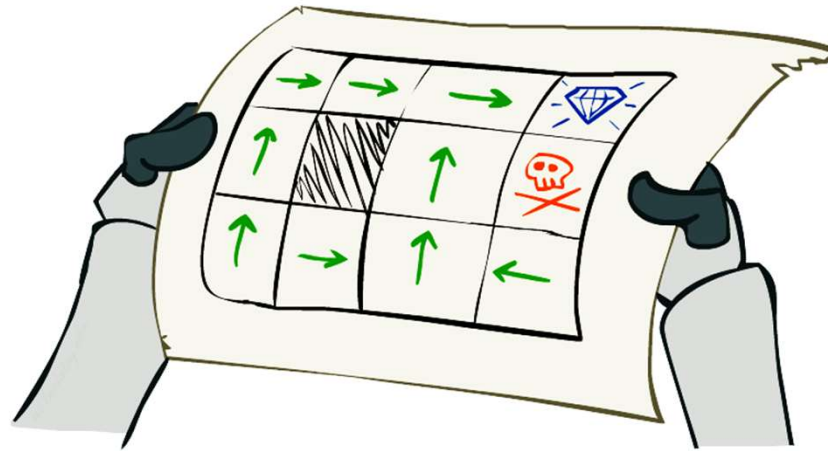
Yiping.fang@centralesupelec.fr



- Multistage stochastic programming
- A different perspective: Markov decision processes
 - modeling elements
 - solving MDP: value iteration and policy iteration
 - dual perspective: linear programming for MDP



- How to find the optimal policy $\pi^* : S \rightarrow A$?



- Model-based methods:
 - Value iteration
 - Policy iteration
 - Linear programming for MDPs
- Model free: reinforcement learning (Q-learning, SARSA, etc.)



- The **optimal value** of a state s

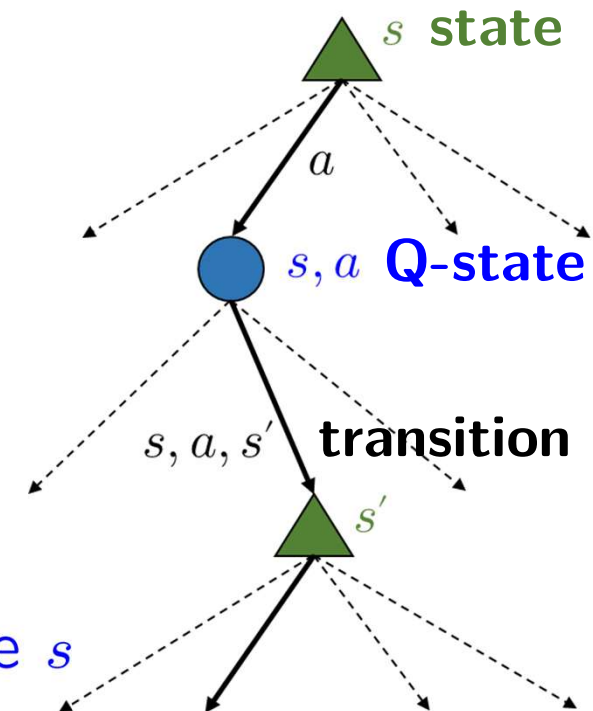
$V^*(s)$ = expected utility starting in s
and acting optimally

- The **optimal value** of a Q-state (s, a)

$Q^*(s, a)$ = expected utility starting out
having taken action a from state s
and (thereafter) acting optimally

- The **optimal policy**

$\pi^*(s)$ = optimal action from state s

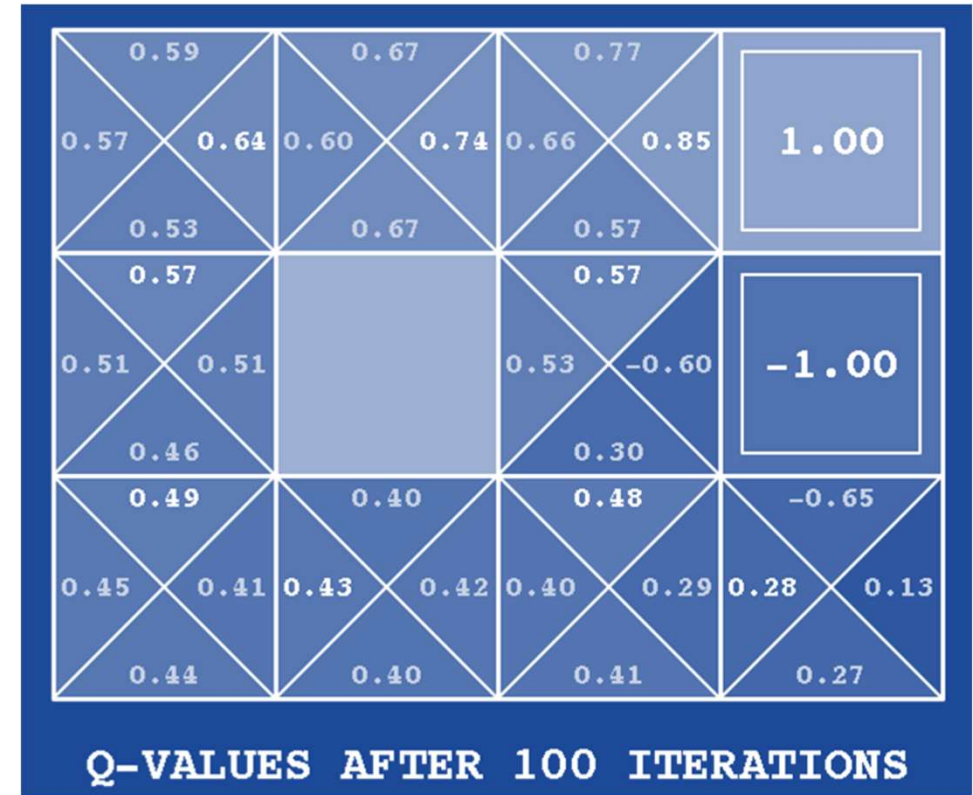
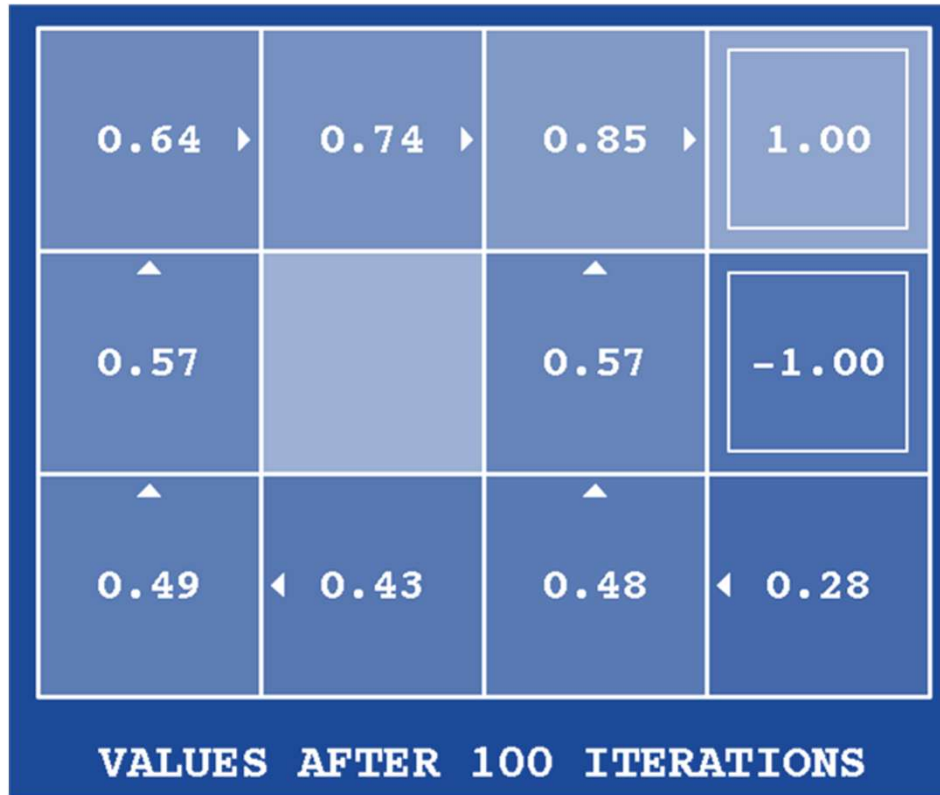




V^* and Q^*



CentraleSupélec



Noise = 0.2

Discount = 0.9

Living reward = 0



- **Fundamental operation:** compute the (expected max) **optimal value** of a state

→ Expected utility under **optimal** action

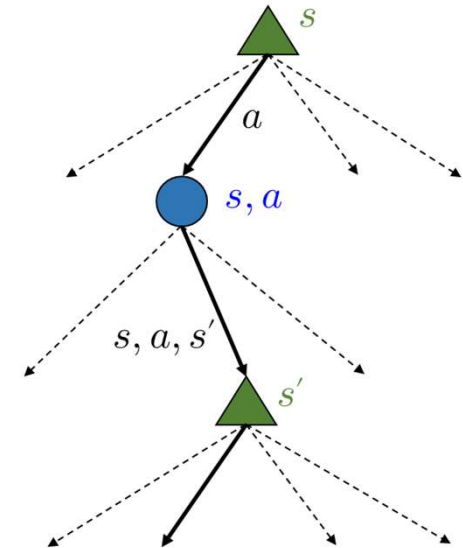
→ Average sum of (discounted) rewards

- **Recursive definition of values**
(Bellman optimality equations)

$$V^*(s) = \max_{a \in A} Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

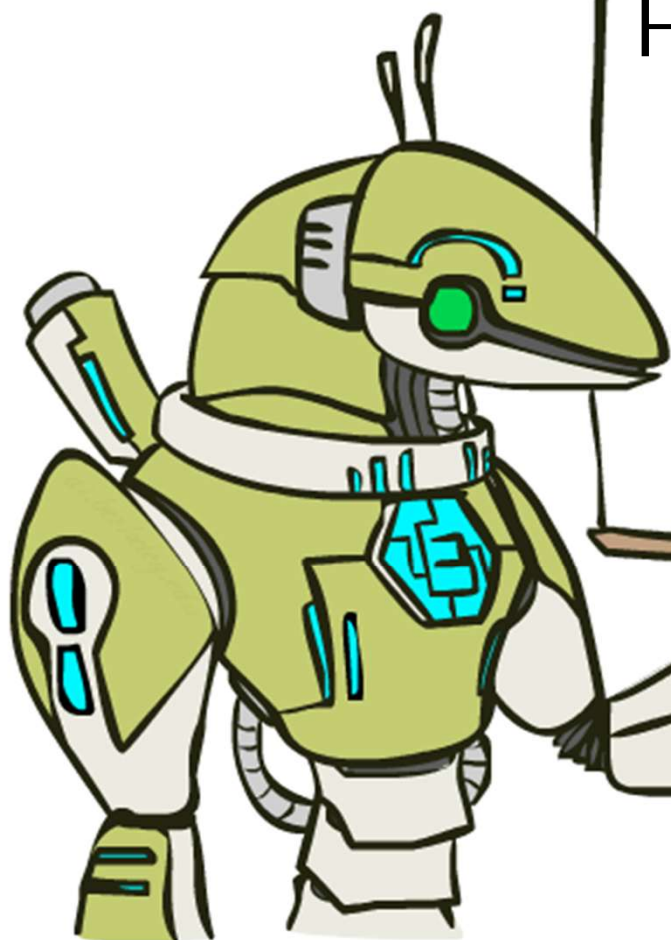




The Bellman equations



CentraleSupélec



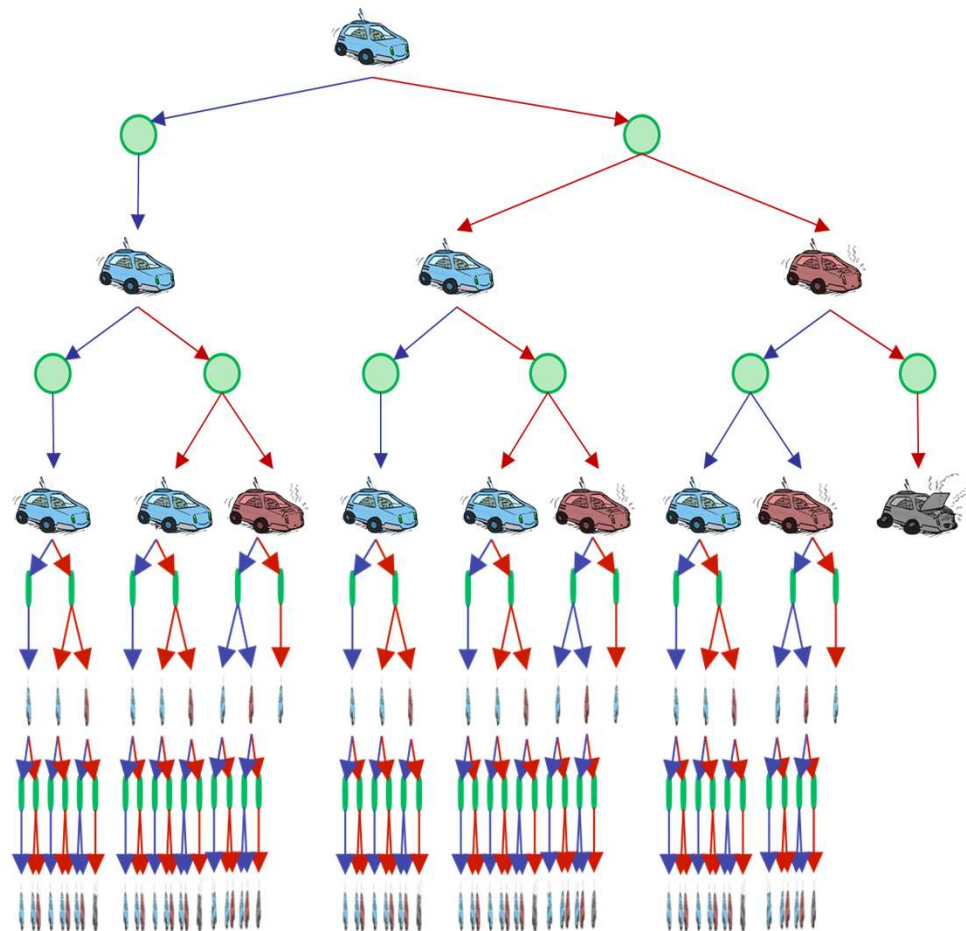
How to be optimal:

Step 1: Take correct first action

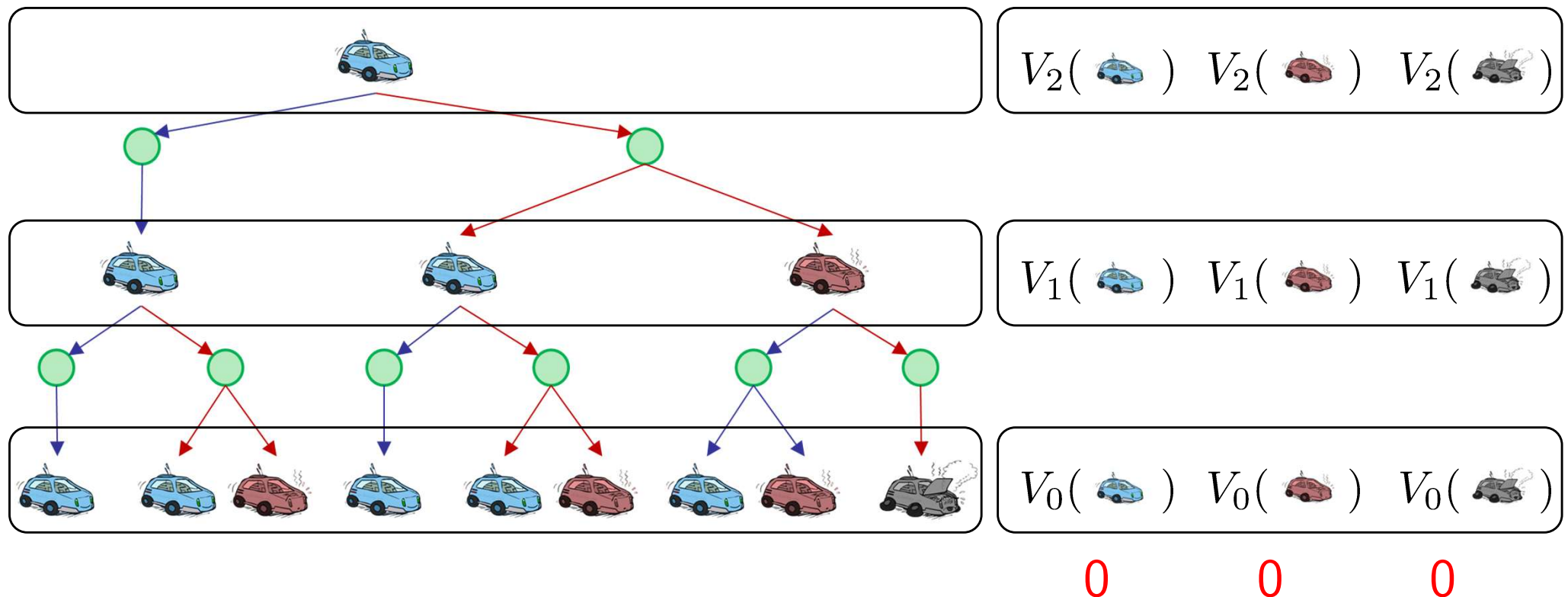
Step 2: Keep being optimal



- Define $V_k(s)$ to be the **optimal value of s** if the decision horizon **ends in k more steps**
- How about infinite horizon (tree goes forever)?
 - idea: do a **depth-limited** computation, but with **increasing depths** until change is small
 - note: deep parts of the tree eventually don't matter if $\gamma < 1$



Decision horizon $K = 2$



$$V^*(s) = \max_{a \in A} \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



- **Idea:** repeatedly update an estimate of the optimal value function according to **Bellman optimality equation**
- **Procedure**
 - 1) Initialize $V_0(s) \leftarrow 0, \forall s \in S$: no time steps left means an expected reward sum of zero
 - 2) Update:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

3) Repeat until convergence

- Complexity of each iteration: $O(|S|2|A|)$



Illustration of value iteration



CentraleSupélec

▲ 0.00	▲ 0.00	▲ 0.00	0.00
▲ 0.00		▲ 0.00	0.00
▲ 0.00	▲ 0.00	▲ 0.00	▲ 0.00
VALUES AFTER 0 ITERATIONS			

$$k = 0$$

Noise = 0.2

Discount = 0.9

Living reward = 0



Illustration of value iteration



CentraleSupélec

▲ 0.00	▲ 0.00	0.00 ▶	1.00
▲ 0.00		◀ 0.00	-1.00
▲ 0.00	▲ 0.00	▲ 0.00	0.00 ▼
VALUES AFTER 1 ITERATIONS			

$$k = 1$$

Noise = 0.2

Discount = 0.9

Living reward = 0



Illustration of value iteration



CentraleSupélec

▲ 0.00	0.00 ▶	0.72 ▶	1.00
▲ 0.00		▲ 0.00	-1.00
▲ 0.00	▲ 0.00	▲ 0.00	0.00 ▼
VALUES AFTER 2 ITERATIONS			

$$k = 2$$

Noise = 0.2

Discount = 0.9

Living reward = 0



Illustration of value iteration



CentraleSupélec

0.00 ▶	0.52 ▶	0.78 ▶	1.00
▲ 0.00		▲ 0.43	▼ -1.00
▲ 0.00	▲ 0.00	▲ 0.00	▼ 0.00
VALUES AFTER 3 ITERATIONS			

$$k = 3$$

Noise = 0.2

Discount = 0.9

Living reward = 0



Illustration of value iteration



CentraleSupélec

0.37 ▶	0.66 ▶	0.83 ▶	1.00
▲ 0.00		▲ 0.51	-1.00
▲ 0.00	0.00 ▶	▲ 0.31	◀ 0.00
VALUES AFTER 4 ITERATIONS			

$k = 4$

Noise = 0.2

Discount = 0.9

Living reward = 0



Illustration of value iteration



CentraleSupélec

0.51 ▶	0.72 ▶	0.84 ▶	1.00
▲ 0.27		▲ 0.55	-1.00
▲ 0.00	0.22 ▶	▲ 0.37	◀ 0.13
VALUES AFTER 5 ITERATIONS			

$$k = 5$$

Noise = 0.2

Discount = 0.9

Living reward = 0



Illustration of value iteration



CentraleSupélec

0.64 ▶	0.74 ▶	0.85 ▶	1.00
▲ 0.56		▲ 0.57	-1.00
▲ 0.48	◀ 0.41	▲ 0.47	◀ 0.27
VALUES AFTER 10 ITERATIONS			

$k = 10$

Noise = 0.2

Discount = 0.9

Living reward = 0



Illustration of value iteration



CentraleSupélec

0.64 ▶	0.74 ▶	0.85 ▶	1.00
▲ 0.57		▲ 0.57	-1.00
▲ 0.49	◀ 0.43	▲ 0.48	◀ 0.28
VALUES AFTER 100 ITERATIONS			

$k = 100$

Noise = 0.2

Discount = 0.9

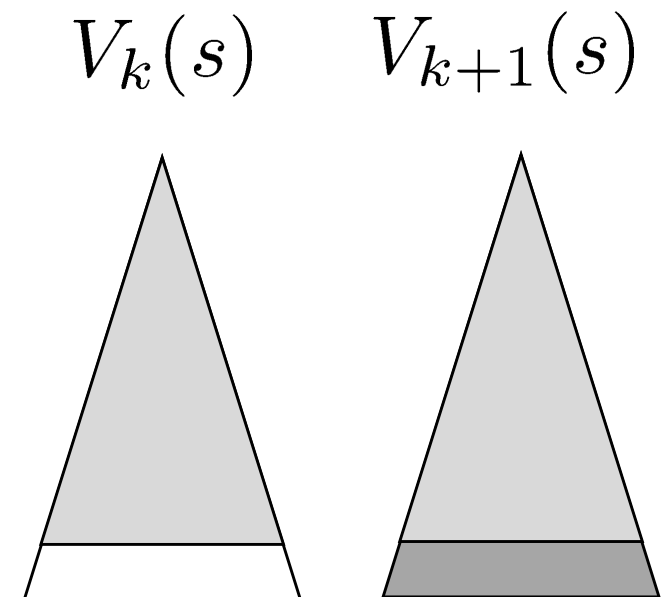
Living reward = 0



Theorem: value iteration asymptotically converges to optimal value $V_k \rightarrow V^*$

Conceptual proof:

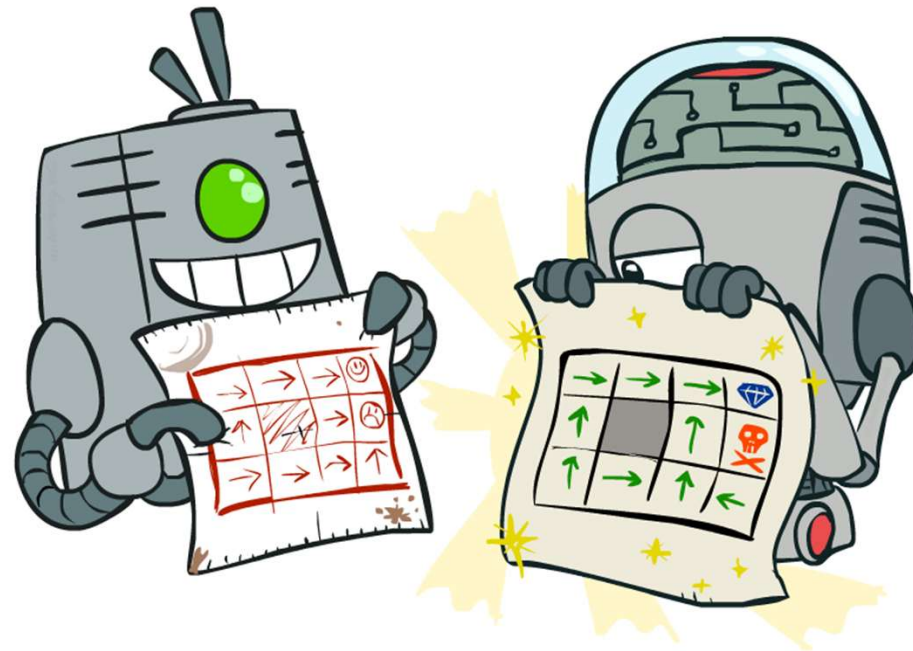
- **Case 1:** if the decision horizon is **finite** with T steps, then, V_T holds the actual untruncated values
- **Case 2:** if infinite horizon with $\gamma < 1$
- **Sketch:** for any state V_k and V_{k+1} can be viewed as depth $k+1$ expected-max results in **nearly identical** search trees
- The difference: on the bottom layer, V_{k+1} has actual rewards while V_k has zeros
- That last layer is at best all R_{MAX} , at worst R_{MIN} ; but everything is discounted by γ^k that far out, so



$$|V_{k+1} - V_k| \leq \gamma^k R_{MAX}$$



- Focus on the quality of a policy (to find the optimal one)
 - Evaluate a policy
 - Greedily improve the policy





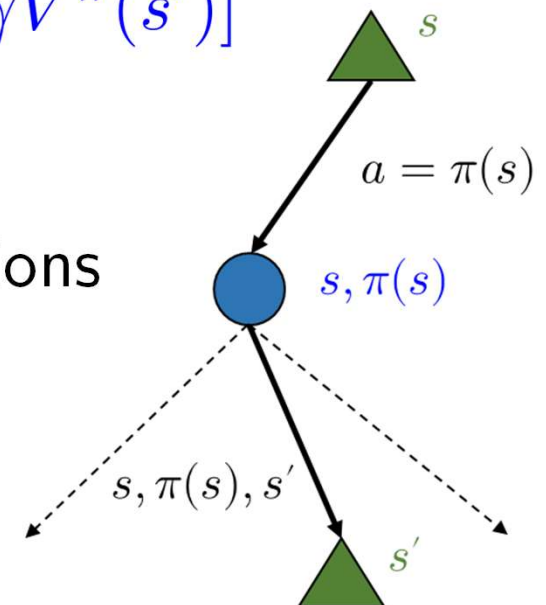
- Calculate the **value function** for a fixed policy π

$$V^\pi = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) | s_0, a_t = \pi(s_t), T(s_t, a_t, s_{t+1}) \right]$$

- Use the one-step look-ahead / Bellman equation:

$$V^\pi(s) = \sum_{s' \in S} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

- Idea 1: without the max, the Bellman equations are just **a linear equation system**



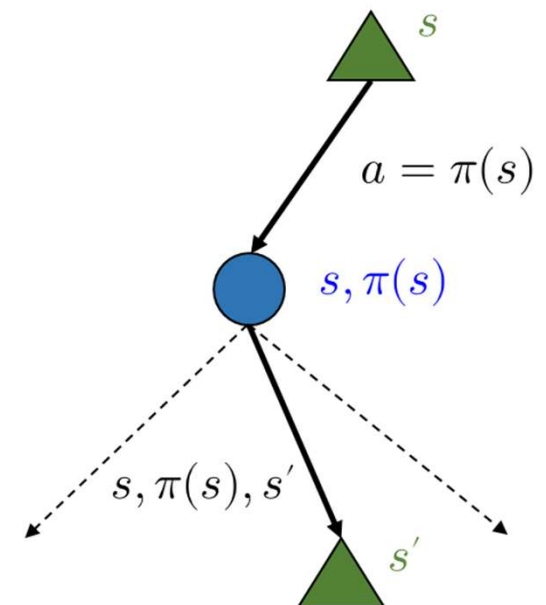


- Idea 2: same to the value iteration, turn recursive Bellman equations into updates:

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- Efficiency: $O(|S|^2)$ per iteration





- Suppose we have the optimal value

$$V^*(s)$$

- How should we make actions? (not obvious!)
- We need to do an one-step expected-max

0.64 ▶	0.74 ▶	0.85 ▶	1.00
▲ 0.57		▲ 0.57	-1.00
▲ 0.49	◀ 0.43	▲ 0.48	◀ 0.28

$$\arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- It is called **policy extraction**, since it gets the policy implied by the state values



Compute actions from Q-values



CentraleSupélec

- Suppose we have the optimal Q-values $Q^*(s, a)$
- How should we make actions?
→ Completely trivial!

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

→ Actions are easier to extract from Q-values than (state) values

0.59	0.67	0.77	1.00
0.57	0.64	0.60	0.74
0.53	0.67	0.57	0.85
0.57		0.57	-1.00
0.51	0.51	0.53	-0.60
0.46		0.30	-0.65
0.49	0.40	0.48	-0.65
0.45	0.41	0.43	0.42
0.40	0.40	0.29	0.28
0.44	0.40	0.41	0.27

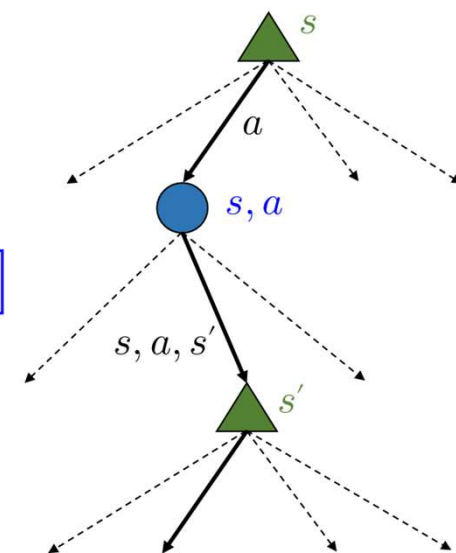


Problem with value iteration

- Value iteration repeat the Bellman updates:

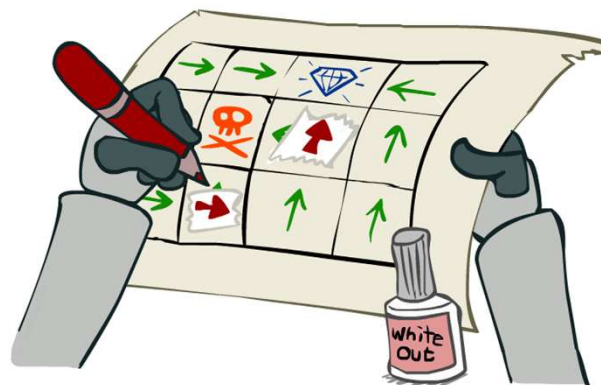
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Issue 1: slow, $O(|S|^2|A|)$ per iteration
- Issue 2: the “max” at each state rarely change
- Issue 3: the policy often converges long before the values



Policy iteration

- Policy evaluation
- Policy improvement





Procedure

- 1) Initialize policy π_0 (e.g., random policy)
- 2) Compute the values of the policy (e.g., solving linear systems)

$$V^{\pi_k}(s) = \sum_{s' \in S} T(s, \pi(s), s') [R(s, \pi_k(s), s') + \gamma V^{\pi}(s')]$$

- 3) Update π_k to be greedy policy with respect to $V^{\pi_k}(s)$

$$\pi_{k+1}(s) \leftarrow \arg \max_a \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_k}(s')]$$

- 4) If $\pi_{k+1} = \pi_k$ terminate; otherwise, return to step 2)



Modified policy iteration procedure

- 1) Initialize policy π_0 (e.g., random policy)
- 2) Compute the values of the policy using recursive iteration,
(approximately) not necessarily converge

$$V_{i+1}^{\pi_k}(s) \leftarrow \sum_{s'} T(s, \pi_k(s), s') [R(s, \pi_k(s), s') + \gamma V_k^{\pi_i}(s')]$$

- 3) Update π_k to be greedy policy with respect to $V^{\pi_k}(s)$

$$\pi_{k+1}(s) \leftarrow \arg \max_a \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_k}(s')]$$

- 4) If $\pi_{k+1} = \pi_k$ terminate; otherwise, return to step 2)

(Putterman and Shin, 1978)



- Both compute the same thing (all optimal values)
- Value iteration:
 - every iter. updates both the values and (**implicitly**) the policy
 - don't track the policy, but taking the max over actions implicitly re-computes it
- Policy iteration:
 - do several passes that **update values with fixed policy** (each pass is fast since we consider only one action)
 - after the policy is evaluated, **a new policy is chosen** (slow like a value iteration pass)
 - the new policy will be better (or we're done)
- Both are **dynamic programs** for solving MDPs



- Multistage stochastic programming
- A different perspective: Markov decision processes
 - modeling elements
 - solving MDP: value iteration and policy iteration
 - dual perspective: linear programming for MDP



- A slightly less frequently described method for MDPs: solution via LP

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- **Basic idea:** consider the inequality form of the Bellman equations which **provide upper bounds** for $V^*(s)$

$$V(s) \geq \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V(s')], \quad \forall a \in A$$

$$\begin{aligned} \Rightarrow V(s) &\geq \max_{a \in A} \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V(s')], \quad \forall s \\ &= \mathcal{H}(V)(s), \quad \forall s \end{aligned}$$

$$\Rightarrow V \geq \mathcal{H}(\mathcal{H}(V)) \geq \dots \geq (\mathcal{H})^\infty(V) = V^*$$



- Now consider the LP

$$\min_{V \in \mathbb{R}^{|S|}} \sum_{s \in S} V(s)$$

$$\text{s.t. } V(s) \geq \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V(s')], \quad \forall a \in A, s \in S$$

- **Theorem:** the optimal value of this LP will be $V^*(s)$
- **Proof:** Suppose there exists some $s \in S$ with

$$V(s) \geq \max_{a \in A} \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

Then we can construct a solution with only $V(s)$ changed to make this an equality: this will have **a lower** objective value, but **be feasible**, since it can only decrease the RHS for other constraints



- In the objective, we can use any positive linear function of $V(s)$ and the result still holds, e.g.,

$$\begin{aligned} \min_{V \in \mathbb{R}^{|S|}} \quad & \sum_{s \in S} w(s) V(s) \\ \text{s.t.} \quad & V(s) \geq \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V(s')], \quad \forall a \in A, s \in S \end{aligned}$$

where $w(s)$ is a distribution over states, initial state distribution

- The objective is the total **expected accumulated rewards** when beginning at a state drawn from this distribution



- LP gives us $V^*(s)$
- We can extract π^* one-step expected-max, as usual:

$$\pi^*(s) \leftarrow \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- Or check the **active constraints**

→ for each s , check which a^* leads to equality

$$V(s) = \sum_{s' \in S} T(s, a^*, s') [R(s, a^*, s') + \gamma V(s')]$$

$$V(s) \geq \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V(s')], \forall a$$

→ set $\pi^*(s) \leftarrow a^*$



- Could we directly optimize π ?
 - Yes, by considering the **dual problem** of the linear program



$$\begin{aligned} \min_{V \in \mathbb{R}^{|S|}} \quad & \sum_{s \in S} w(s) V(s) \\ \text{s.t.} \quad & V(s) \geq \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V(s')], \quad \forall a \in A, s \in S \\ & \lambda(s, a) \end{aligned}$$

- The dual problem

$$\begin{aligned} \max_{\lambda \in \mathbb{R}^{|S| \times |A|}} \quad & \sum_{s \in S, a \in A} \lambda(s, a) \sum_{s' \in S} T(s, a, s') R(s, a, s') \\ \text{s.t.} \quad & \sum_{a \in A} \lambda(s, a) = w(s) + \gamma \sum_{a \in A, s' \in S} T(s', a, s) \lambda(s', a), \quad \forall s \in S \\ & \lambda(s, a) \geq 0 \quad \forall s \in S, a \in A \end{aligned}$$



$$\begin{aligned} \max_{\lambda} \quad & \sum_{s \in S, a \in A} \lambda(s, a) \sum_{s' \in S} T(s, a, s') R(s, a, s') \\ \text{s.t.} \quad & \sum_{a \in A} \lambda(s, a) = w(s) + \gamma \sum_{a \in A, s' \in S} T(s', a, s) \lambda(s', a), \quad \forall s \in S \\ & \lambda(s, a) \geq 0 \quad \forall s \in S, a \in A \end{aligned}$$

- Dual variables have explicit meaning: **discounted Q-state** (s, a) **frequency** (proportional to π)

$$\lambda(s, a) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s, a_t = a)$$

- The **objective** implicitly measures the value of the optimal policy (expected discounted utility)



- Let $f(s)$ be the **discounted frequency of s under policy π**

- Step 0 $f_0(s) = w(s)$

- Step 1 $f_1(s') = w(s') + \gamma \sum_s T(s, \pi(s), s') w(s)$

- Step 2
$$\begin{aligned} f_2(s'') &= w(s'') + \gamma \sum_{s'} T(s', \pi(s'), s'') w(s') \\ &\quad + \gamma^2 \sum_{s, s'} T(s', \pi(s'), s'') T(s, \pi(s), s') w(s) \\ &= w(s'') + \gamma \sum_{s'} T(s', \pi(s'), s'') \\ &\quad \cdot \left\{ w(s') + \gamma \sum_s T(s, \pi(s), s') w(s) \right\} \end{aligned}$$

- Step n

$$f_n(s^{(n)}) = w(s^{(n)}) + \gamma \sum_{s^{(n-1)}} T(s^{(n-1)}, \pi(s^{(n-1)}), s^{(n)}) f_{n-1}(s^{(n-1)})$$

- ∞ steps $f(s) = w(s) + \gamma \sum_{s'} T(s', \pi(s'), s) f(s')$



- Let $\lambda(s, a)$ be the **discounted Q-state (s, a) frequency under policy π**

$$\lambda(s, a) = f(s)\pi(a|s)$$

where $\pi(a|s) = \mathbb{P}(a|s)$ is a stochastic policy

- Then the following equations are equivalent:

$$f(s) = w(s) + \gamma \sum_{s'} T(s', \pi(s'), s) f(s')$$

$$\Leftrightarrow \sum_a \pi(a|s) f(s) = w(s) + \gamma \sum_{s', a} T(s', a, s) \pi(a|s') f(s')$$

$$\Leftrightarrow \underbrace{\sum_a \lambda(s, a) = w(s) + \gamma \sum_{s', a} T(s', a, s) \lambda(s', a)}$$

Constraint of dual LP



- We can recover π from $\lambda(s, a)$ for stochastic policy

$$\lambda(s, a) = f(s)\pi(a|s)$$

$$\pi(a|s) = \frac{\lambda(s, a)}{f(s)}$$

$$\pi(a|s) = \frac{\lambda(s, a)}{\sum_a \lambda(s, a)}$$

- and for deterministic policy

$$\pi^*(s) = \arg \max_{a \in A} \lambda(s, a)$$



- Some **surprising connections** between LP formulation and standard value & policy iteration: e.g. a certain form of dual simplex **is equivalent** to policy iteration
- Typically, best specialized MDP algorithms (e.g. modified policy iteration) **are faster** than general LP algorithms, but the LP formulation provides
 - a natural way to model **constrained MDP**, and
 - easily **comprehensible** with an optimization background (single numerical objective)
 - **powerful tool** for **developing algorithms**, has also been the basis for much work in **approximate large-scale MDP solutions** (e.g., de Farias and Van Roy, 2003)

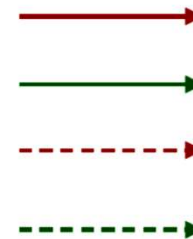
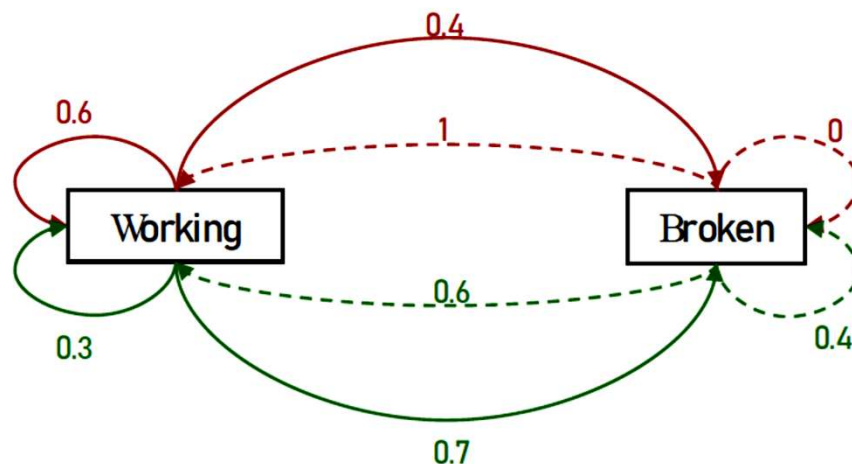


Exercise: maintenance optimization



CentraleSupélec

- Machine: running or broken
- If it runs throughout one month, it makes a profit $q = \$100 K$. If it fails during the month, gross profit is $q = 0$



States	Actions	$C(\text{action})$
Working	Maintenance	20
	Wait	0
Broken	Replace	150
	Repair	40

- Direct reward:

$$R(s, a, s') = -C(a) + q(s')$$

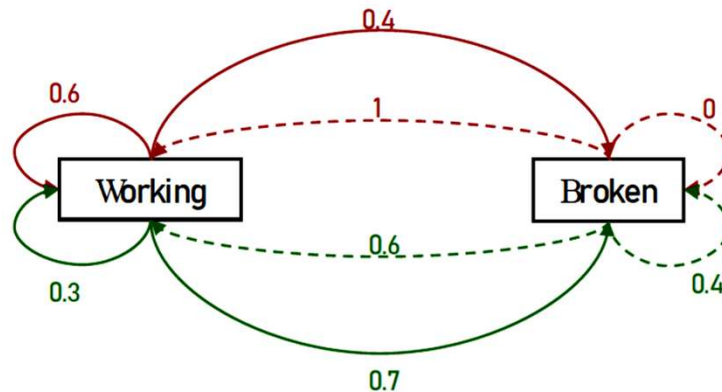
$$R(s, a) = -C(a) + \sum_{s'} T(s, a, s') q(s')$$

- Goal: find the optimal policy for $\gamma = 0.8$



Exercise: maintenance optimization

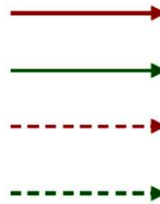
$$V^*(s) = \max_{a \in A} \sum_{s' \in S} T(s, a, s') [-C(a) + q(s') + \gamma V^*(s')]$$



States	Actions	$C(\text{action})$
Working	Maintenance	20
	Wait	0
Broken	Replace	150
	Repair	40

$$V^*(W) = \max \left\{ \underbrace{-20 + 0.6(100 + 0.8V^*(W)) + 0.4(0 + 0.8V^*(B))}_{\text{maintenance}}, \right. \\ \left. \underbrace{0 + 0.3(100 + 0.8V^*(W)) + 0.7(0 + 0.8V^*(B))}_{\text{wait}} \right\}$$

$$V^*(B) = \max \left\{ \underbrace{-150 + 1.0(100 + 0.8V^*(W))}_{\text{replace}}, \right. \\ \left. \underbrace{-40 + 0.6(100 + 0.8V^*(W)) + 0.4(0 + 0.8V^*(B))}_{\text{repair}} \right\}$$



States	Actions	$C(\text{action})$
Working	Maintenance	20
	Wait	0
Broken	Replace	150
	Repair	40

$$V^*(W) = \max \left\{ \underbrace{0.48V^*(W) + 0.32V^*(B) + 40}_{\text{maintenance}}, \underbrace{0.24V^*(W) + 0.56V^*(B) + 30}_{\text{wait}} \right\}$$

$$V^*(B) = \max \left\{ \underbrace{0.8V^*(W) - 50}_{\text{replace}}, \underbrace{0.48V^*(W) + 0.32V^*(B) + 20}_{\text{repair}} \right\}$$

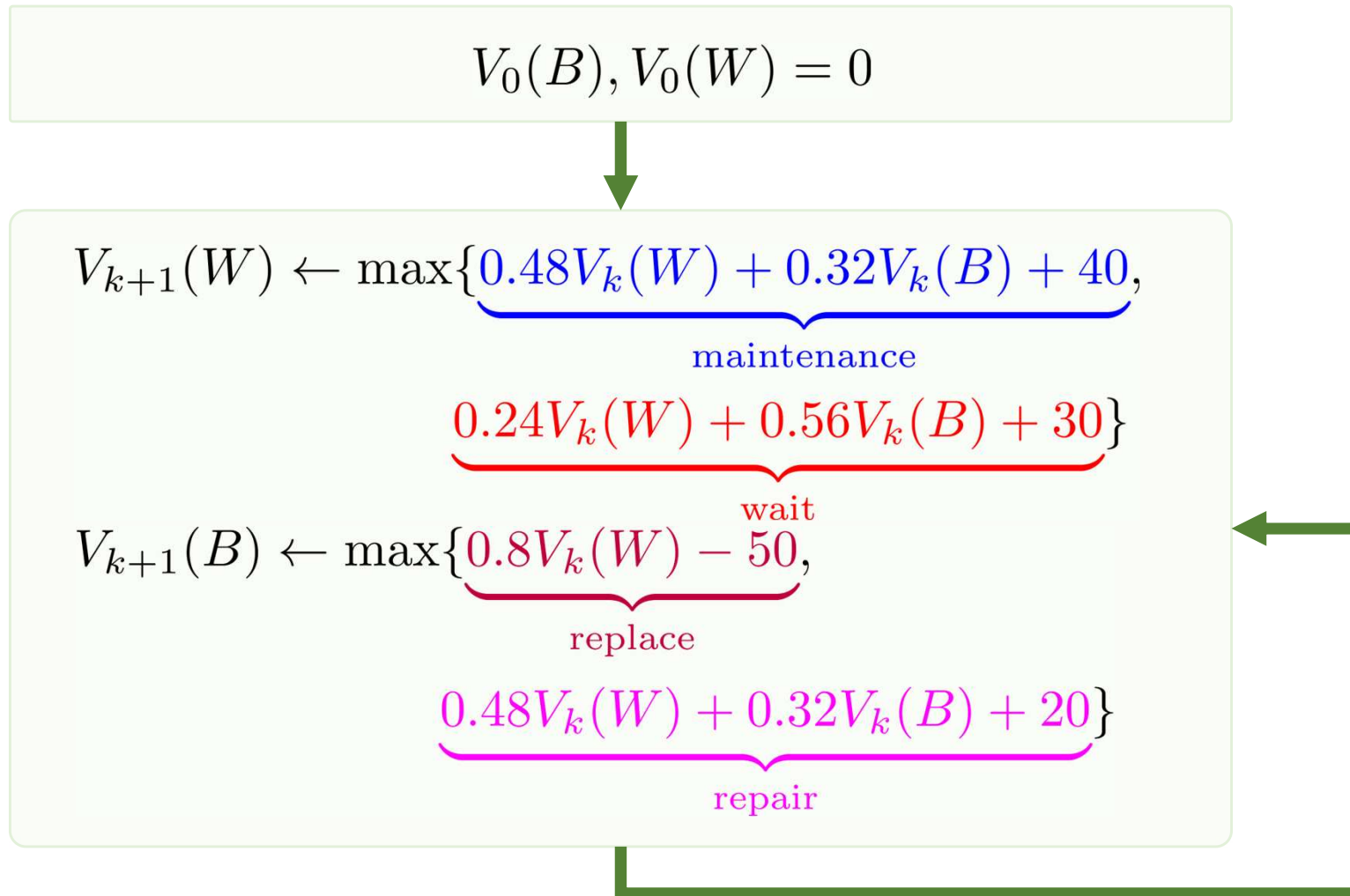


Exercise: maintenance optimization



CentraleSupélec

- Value iteration





Exercise: maintenance optimization



CentraleSupélec

- Policy iteration

$$\pi(W) = \text{wait}, \pi(B) = \text{repair}$$

$$V(W) = \underbrace{0.24V(W) + 0.56V(B) + 30}_{\text{wait}}$$

$$V(W) = 127.419$$

$$V(B) = \underbrace{0.48V(W) + 0.32V(B) + 20}_{\text{repair}}$$

$$V(B) = 119.355$$

$$\pi(W) \leftarrow \arg \max \left\{ \underbrace{0.48V(W) + 0.32V(B) + 40}_{\text{maintenance}}, \underbrace{0.24V(W) + 0.56V(B) + 30}_{\text{wait}} \right\}$$

$$\left\{ \underbrace{139.355}_{\text{maintenance}}, \underbrace{127.419}_{\text{wait}} \right\}$$

$$\left\{ \underbrace{51.935}_{\text{replace}}, \underbrace{119.355}_{\text{repair}} \right\}$$

$$\pi(B) \leftarrow \arg \max \left\{ \underbrace{0.8V(W) - 50}_{\text{replace}}, \underbrace{0.48V(W) + 0.32V(B) + 20}_{\text{repair}} \right\}$$



Exercise: maintenance optimization



CentraleSupélec

- Linear programming, primal problem

$$\min \quad V(W) + V(B)$$

$$\text{s.t.} \quad V(W) \geq 0.48V(W) + 0.32V(B) + 40$$

$$V(W) \geq 0.24V(W) + 0.56V(B) + 30$$

$$V(B) \geq 0.8V(W) - 50$$

$$V(B) \geq 0.48V(W) + 0.32V(B) + 20$$

$$\max \quad 40\lambda(W, Mt) + 30\lambda(W, Wt) - 50\lambda(B, Re) + 20\lambda(B, Rr)$$

$$\text{s.t.} \quad \lambda(W, Mt) + \lambda(W, Wt) = 1 +$$

$$0.8 \{0.6\lambda(W, Mt) + 0.3\lambda(W, Wt) + 1\lambda(B, Re) + 0.6\lambda(B, Rr)\}$$

$$\lambda(B, Re) + \lambda(B, Rr) = 1 +$$

$$0.8 \{0\lambda(B, Re) + 0.4\lambda(B, Rr) + 0.4\lambda(W, Mt) + 0.7\lambda(W, Wt)\}$$

$$\lambda \geq 0$$



Exercise: maintenance optimization



CentraleSupélec

- Solution

$$\left\{ \begin{array}{lcl} \lambda(W, Mt) & = & 5.8 \\ \lambda(W, Wt) & = & 0 \\ \lambda(B, Re) & = & 0 \\ \lambda(B, Rr) & = & 4.2 \end{array} \right. \longrightarrow \left\{ \begin{array}{lcl} \pi^*(W) & = & Mt \\ \pi^*(B) & = & Rr \end{array} \right.$$



路漫漫其修远兮
吾将上下而求索

屈原 離騷



yiping.fang@centralesupelec.fr